

Predictive Modeling for Heart Disease

End-to-End Data Science & Machine Learning Implementation Blueprint

Data Science

Scikit-Learn

TensorFlow / Keras

Antigravity IDE Ready

1. Executive Project Overview

This project builds an automated, end-to-end binary classification pipeline to predict cardiovascular disease risk using clinical parameters. The system compares a hyperparameter-tuned **Random Forest Classifier** against a deep learning **Artificial Neural Network (ANN)** to evaluate predictive capability across clinical metrics including Precision, Recall, and ROC-AUC.

2. Clinical Dataset Architecture

The model processes 14 foundational clinical attributes (commonly derived from the UCI Cleveland Dataset):

Attribute	Description	Type / Values
age, sex	Age in years, Biological sex	Continuous, Binary (1 = Male, 0 = Female)
cp	Chest Pain Type	Categorical (0: Typical, 1: Atypical, 2: Non-anginal, 3: Asymptomatic)
trestbps	Resting Blood Pressure	Continuous (mm Hg on admission)
chol	Serum Cholesterol	Continuous (mg/dl)
fb	Fasting Blood Sugar > 120 mg/dl	Binary (1 = True, 0 = False)
restecg	Resting Electrocardiographic Results	Categorical (0: Normal, 1: ST-T wave abnormality, 2: LV hypertrophy)
thalach	Maximum Heart Rate Achieved	Continuous
exang	Exercise Induced Angina	Binary (1 = Yes, 0 = No)
oldpeak, slope	ST Depression & Peak Exercise ST Slope	Continuous, Categorical (0: Upsloping, 1: Flat, 2: Downsloping)
ca, thal	Major Vessels Colored (0-3) & Thalassemia	Discrete Integer, Categorical (1: Normal, 2: Fixed defect, 3: Reversible)
target	Heart Disease Diagnosis (Ground Truth)	Binary (0 = Absence, 1 = Presence)

3. End-to-End ML Pipeline Workflow

- **Data Ingestion & Synthesis:** Loads clinical dataset or synthetically generates correlated clinical features if local data is unavailable.
- **Exploratory Data Analysis (EDA):** Visualizes target distribution, missingness, and correlation matrix heatmap.

- **Preprocessing & Normalization:** Stratified 80/20 train/test split, feature scaling using StandardScaler, and SMOTE oversampling for class balance.
- **Model A - Random Forest:** Ensembled trees optimized via GridSearchCV across n_estimators, max_depth, and min_samples_split.
- **Model B - Artificial Neural Network (ANN):** Keras sequential multi-layer perceptron with Dense layers, Batch Normalization, Dropout (0.2), ReLU activation, Sigmoid output, and EarlyStopping.
- **Evaluation & Diagnostics:** Generates side-by-side Confusion Matrices, Classification Reports (Precision, Recall, F1), and overlaid ROC-AUC curves.

4. Antigravity IDE Master Prompt

Copy and paste the exact prompt below directly into **Antigravity IDE** to automatically generate the full project codebase:

PROMPT FOR ANTIGRAVITY IDE

Act as an expert Data Scientist and Machine Learning Engineer. I want to build a complete Python project for "Predictive Modeling for Heart Disease Risk".

Please generate a structured, production-ready Python codebase using Scikit-Learn, TensorFlow/Keras, Pandas, NumPy, and Matplotlib/Seaborn.

Requirements:

1. Data Loading & Synthetic Dataset Generation:

- Create a helper function that either downloads/loads the UCI Heart Disease dataset or synthetically generates a realistic 1,000-row DataFrame with standard clinical features (age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal, target) if no CSV path is provided.

2. Exploratory Data Analysis (EDA):

- Include code to display basic statistics, check for nulls, and save/plot a Correlation Heatmap and Target Distribution plot.

3. Preprocessing Pipeline:

- Split dataset into train/test sets (80/20 split, stratified).
- Apply StandardScaler to continuous numerical features.
- Handle potential class imbalance using SMOTE (from imblearn).

4. Model Building & Hyperparameter Tuning:

- Model A: Random Forest Classifier tuned using GridSearchCV for optimal max_depth, n_estimators, and min_samples_split.
- Model B: Artificial Neural Network (ANN) using Keras with:
 - * Dense input layer (ReLU activation, Dropout 0.2)
 - * Hidden layer (Dense ReLU, Batch Normalization)
 - * Output layer (Dense Sigmoid for binary classification)
 - * Compiled with Adam optimizer and binary_crossentropy loss. Include EarlyStopping.

5. Evaluation & Comparison:

- Print classification report (Precision, Recall, F1-Score) for both models.
- Plot ROC-AUC curves comparing Random Forest vs. ANN on the test set.
- Plot Confusion Matrices for both models.

6. Code Structure:

- Format the code cleanly into functions or modular script files with clear inline comments.

5. Implementation Checklist & Environment Setup

Required Python Dependencies:

```
pip install pandas numpy scikit-learn tensorflow imbalanced-learn matplotlib seaborn
```